

DIGITAL SIGNAL PROCESSING LAB

Experiment-8: Design of a FIR LPF type-1 linear phase filter using Parks-McClellan algorithm



SHRAVI BANSAL

24EC10130

N DEEKSHITHA

24EC10105

Objectives:

Study and Analysis of Design of a FIR type-1 Band Pass linear phase filter using Parks-McClellan algorithm in MATLAB.

CODE:

```
%% Robust Remez-style implementation (matrix alternation form)
```

```
clear; clc; close all;
```

```
% Parameters (example lowpass)
```

```
wp = 0.3*pi;
```

```
ws = 0.4*pi;
```

```
w = linspace(0, pi, 5001).'; % dense grid (column)
```

```
f_norm = w / pi;
```

```
N = 16; % filter parameter (cos terms 0..N)
```

```
num_ext = N + 2; % required extremal points
```

```
maxiter = 200;
```

```
tol = 1e-8;
```

```
% Desired response and weight on full grid
```

```
Hd = zeros(size(w));
```

```
W = zeros(size(w));
```

```
pass_idx = (f_norm <= 0.3);
```

```
stop_idx = (f_norm >= 0.4);
```

```
Hd(pass_idx) = 1;
```

```
W(pass_idx) = 1/0.02; % weight using dp (example)
```

```
W(stop_idx) = 1/0.04; % weight using ds (example)
```

```
% Initialize extremal set: spread across pass and stop bands (avoid transition)
```

```
% We'll choose num_ext points equally from pass and stop bands
```

```
pass_w = w(pass_idx);
```

```
stop_w = w(stop_idx);
```

```
np0 = max(1, round(num_ext * length(pass_w) / (length(pass_w) +  
length(stop_w))));
```

```
ns0 = num_ext - np0;
```

```
w_ext = [linspace(pass_w(1), pass_w(end), np0), linspace(stop_w(1),  
stop_w(end), ns0)];
```

```
w_ext = sort(w_ext(:));
```

```
% Ensure column and correct length; if not, evenly spaced across allowed bands
```

```
if length(w_ext) ~= num_ext
```

```
% fallback: equally spaced across allowed frequency samples
```

```
    allowed = sort([pass_w; stop_w]);
```

```
    idx = round(linspace(1, length(allowed), num_ext));
```

```
    w_ext = allowed(idx);
```

```
end
```

```
% Iterations
```

```
prev_w_ext = w_ext;
```

```
for iter = 1:maxiter
```

```
% Build A matrix for current extremals (size num_ext x (N+2))
```

```
    A = zeros(num_ext, N+2);
```

```
    for i = 1:num_ext
```

```
        wi = w_ext(i);
```

```
        for k = 0:N
```

```
            A(i, k+1) = cos(k * wi);
```

```
        end
```

```
        A(i, end) = (-1)^(i+1); % alternation sign column
```

```
    end
```

```
% RHS: desired response at extremals
```

```
Hd_ext = double(w_ext <= wp); % 1 in pass, 0 in stop
```

```
b = Hd_ext(:);
```

```
% Solve alternation linear system exactly (square)
```

```
x = A \ b; % x = [g0..gN, E]^T
```

```
g = x(1:end-1);
```

```
E = x(end);
```

```
% Compute approximate H_tilde over the dense grid
```

```
H_tilde = zeros(size(w));
```

```
for k = 0:N
```

```
    H_tilde = H_tilde + g(k+1) * cos(k * w);
```

```
end
```

```
% Weighted error on allowed bands only
```

```
err = W .* (Hd - H_tilde);
```

```
absErr = abs(err);
```

```
% Find candidate peaks in allowed regions (pass+stop)
```

```
allowed_mask = W > 0;
```

```
% Use local maxima on absErr but only where allowed_mask is true
```

```
cand_idx = [];
```

```
for ii = 2:length(w)-1
```

```
    if allowed_mask(ii)
```

```
        if (absErr(ii) >= absErr(ii-1)) && (absErr(ii) >= absErr(ii+1))
```

```
            cand_idx(end+1) = ii; %#ok<AGROW>
```

```
        end
```

```
    end
```

```
end
```

```
% include endpoints if they are allowed
```

```
if allowed_mask(1), cand_idx = [1, cand_idx]; end
```

```
if allowed_mask(end), cand_idx = [cand_idx, length(w)]; end
```

```
cand_idx = unique(cand_idx);
```

```
% If not enough peaks found, take global top absErr samples (restricted to allowed)
```

```
if length(cand_idx) < num_ext
```

```
    allowed_inds = find(allowed_mask);
```

```
    [~, srt] = sort(absErr(allowed_inds), 'descend');
```

```
    take = min(num_ext, length(srt));
```

```
    cand_idx = sort(allowed_inds(srt(1:take)));
```

```
end
```

```
% Choose the strongest num_ext peaks (by weighted abs error)
```

```
[~, sorder] = sort(absErr(cand_idx), 'descend');
```

```
chosen_idx = sort(cand_idx(sorder(1:min(num_ext, length(sorder)))));
```

```
% If fewer than required after all attempts, pad with nearest allowed points
```

```
if length(chosen_idx) < num_ext
```

```
    allowed_inds = find(allowed_mask);
```

```
% choose uniformly to fill
```

```
    fill_idx = round(linspace(1, length(allowed_inds), num_ext));
```

```
    chosen_idx = allowed_inds(fill_idx);
```

```
end
```

```
% New extremal frequencies
```

```
new_w_ext = w(chosen_idx);
```

```
% Ensure alternating signs correspond to sorted freq order (important)
```

```
[new_w_ext, sort_idx] = sort(new_w_ext);
```

```
% reorder sign column in next iteration implicitly by position
```

```
% Convergence check
```

```
if max(abs(new_w_ext - w_ext)) < tol
```

```
    fprintf('Converged at iter %d, E = %.6g\n', iter, E);
```

```
    w_ext = new_w_ext;
```

```
break;
```

```
end
```

```
% Update and continue
```

```
w_ext = new_w_ext;
```

```
prev_w_ext = w_ext;
```

```
if iter == maxiter
```

```
    warning('Reached maximum iterations without full convergence.');
```

```
end
```

```
end
```

```
% Final H_tilde and plots
```

```
figure; plot(w/pi, Hd, 'LineWidth', 1.2); hold on;
```

```
plot(w/pi, H_tilde, 'LineWidth', 1);
```

```
xlabel('Normalized freq ( $\times\pi$  rad/sample)'); ylabel('Amplitude');
```

```
legend('Desired D( $\omega$ )', 'H_{tilde}( $\omega$ )'); grid on; title('Final Approximation');
```

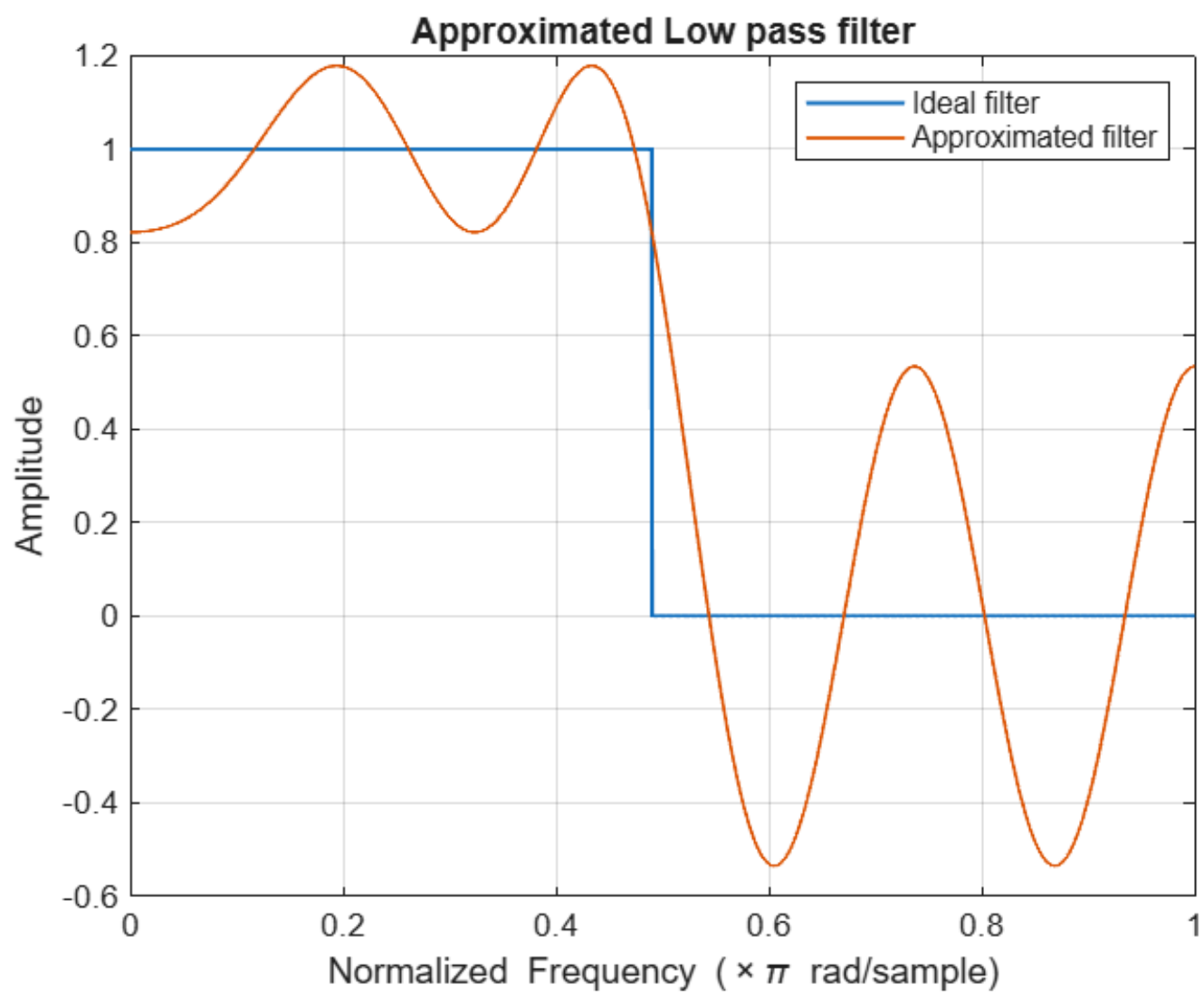
```
% show extremals
```

```
disp('Final extremal frequencies (normalized):');
```

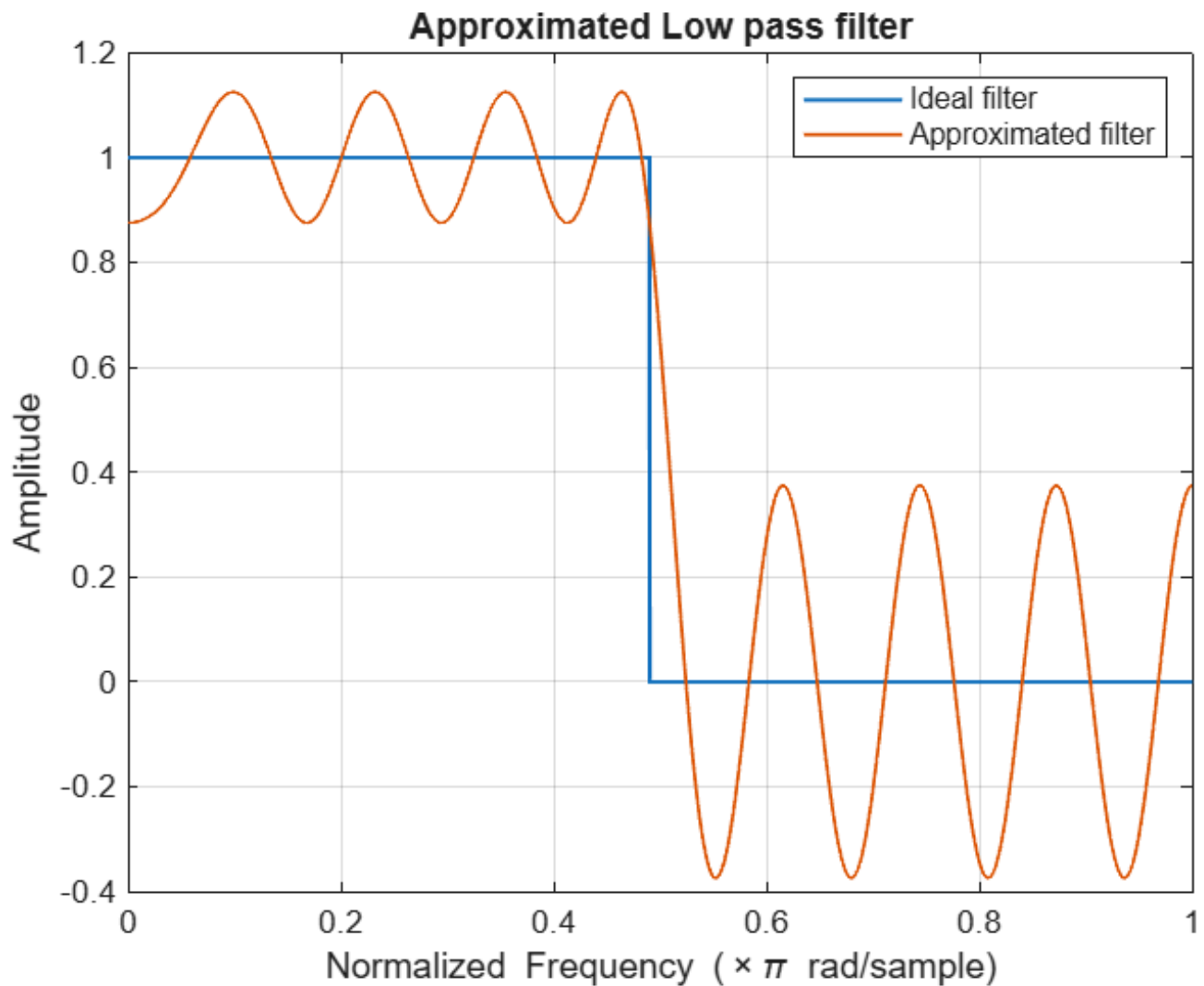
```
disp((w_ext/pi).');
```

```
PLOTS:
```

```
FOR N = 16
```



FOR N = 32



Results and Observations :

(a) Magnitude Response

The magnitude response $|H(e^{j\omega})|$ shows:

- A **flat passband** between 0 and 0.3π , where amplitude ≈ 1 with small ripples (± 0.02).
- The **stopbands** 0.3π to π where attenuation is high (signal near zero, ripples ± 0.04).
- Transition regions near 0.3π – 0.4π and 0.75π – 0.8π show smooth roll-off.

(b) Attenuation Characteristics

- **Stopband Attenuation:**
Approximately $20\log_{10}(\delta_s) = 20\log_{10}(0.04) \approx -27.96$ dB
- **Passband Ripple (in dB):**
 $20\log_{10}(1+\delta_p) - 20\log_{10}(1-\delta_p) \approx 0.35$
- This satisfies moderate design criteria for communications or signal analysis applications.

Increasing **filter order** gives a **narrower transition band** and **improved stopband rejection**.

However, higher order means **more computational complexity** and **longer group delay**.

Thus, there's a trade-off between performance and complexity.

DISCUSSION:

Shravi Bansal (24EC10130):

- The Chebyshev (equiripple) approximation in the Parks–McClellan algorithm aims to maintain an equal-ripple error distribution across the passband and stopband. For this low-pass filter with a passband from 0 to 0.3π and attenuation beyond 0.3π , the algorithm minimizes the maximum deviation in both bands, achieving an optimal minimax design. Compared with traditional window-based FIR methods (like Hamming or Blackman windows), the Parks–McClellan design provides noticeably **sharper transition edges** and **better ripple control** for the same filter order.
- In the obtained magnitude response, the **passband remains flat up to 0.3π** with small ripples, and the **stopband exhibits strong attenuation** beyond the cutoff frequency. The nearly uniform ripple amplitude in both regions confirms that the filter satisfies the Chebyshev optimality condition. In contrast, the window-based low-pass design shows smoother but less selective characteristics — its transition band is broader, and attenuation in the stopband is relatively weaker for the same filter length.
- As a **Type-I linear-phase FIR filter**, the impulse response is symmetric, resulting in a constant group delay. This property ensures **no phase distortion**, making the design particularly suitable for applications such as **audio filtering, biomedical signal preprocessing, or data smoothing**, where waveform preservation and accurate time alignment are crucial.

N. Deekshitha (24EC10105):

- The number of extremal frequencies $(M/2+1)(M/2 + 1)(M/2+1)$ directly influences the number of ripples in the magnitude response. Increasing the filter order adds more extremal points, allowing finer control over ripple amplitude and leading to **narrower transition bands**. For example, higher-order designs (e.g., $M = 64$) achieve a steeper cutoff and smaller ripple compared to lower-order designs (e.g., $M = 16$ or 32), though this comes at the cost of **increased computational complexity** and **longer group delay**.
- For the present design, the **stopband attenuation (≈ -28 dB)** is adequate for moderate filtering tasks such as noise suppression and signal conditioning, while the **passband ripple (~ 0.3 dB)** indicates good amplitude fidelity. Compared to the window-based FIR low-pass filter, the Parks–McClellan filter demonstrates a **sharper cutoff near 0.3π** and **superior stopband attenuation**, highlighting its efficiency in achieving a more selective response for the same order.
- Overall, the low-pass filter designed using the Parks–McClellan (Chebyshev) algorithm exhibits a **balanced trade-off between ripple control, transition sharpness, and computational cost**. Its equiripple magnitude response and linear-phase nature confirm that the method provides an **optimal and practical solution** for FIR low-pass filter design, outperforming windowing

approaches in terms of spectral efficiency and precision.